ESP32-CAM Security Robot

Project Development & Upgrade Documentation

Version 1.0 → Portfolio-Grade Upgrade Plan

Author: Inosuke | Platform: ESP32-CAM AI Thinker | April 2026

1. Current System Overview

1.1 What This Project Is

The ESP32-CAM Security Robot is a WiFi-connected mobile surveillance platform built on the ESP32-CAM AI Thinker module. It combines real-time MJPEG video streaming, browser-based motor control, ultrasonic obstacle avoidance, and on-chip face detection with buzzer alerting — all in a single embedded system with no external compute host required.

1.2 Current Capabilities

- Live MJPEG camera stream served over a local web page
- Motor control via HTTP commands through L298N driver (4 GPIO pins for 2 DC motors)
- Ultrasonic distance sensing using HC-SR04 on GPIO2/GPIO16
- Active buzzer alarm on GPIO4 triggered on face detection events
- ESP32 on-chip face detection using the built-in ESP-WHO library
- Simple obstacle avoidance: stops and alerts when distance falls below threshold

1.3 Current Limitations

- Face detection — not recognition: any face triggers the alarm, with no identification or memory
- No event logging: detected faces and distance events are not saved anywhere
- No remote notifications: alerts stay local; no Telegram, email, or mobile push
- HC-SR04 limitations: narrow beam (~15°), susceptible to soft surfaces and angles, minimum range ~2 cm, max ~4 m reliable
- Single ultrasonic sensor: no side or rear obstacle awareness
- No pan/tilt control: camera points in one fixed direction
- No autonomous patrol: navigation is fully manual via browser
- No snapshot capture: when face detected, no image is saved
- L298N inefficiency: causes significant voltage drop (~2V) and heat generation
- Power: likely direct USB or unregulated battery supply; no monitoring or management
- GPIO16 for ECHO: requires voltage divider (ESP32 is 3.3V tolerant); potential instability at speed

1.4 Architectural Bottlenecks

Memory Constraint

The ESP32-CAM has approximately 520KB of internal SRAM + 4MB PSRAM. Running face detection, MJPEG streaming, motor control, and WiFi simultaneously pushes memory boundaries. Frame buffer allocation and detection buffers compete for PSRAM.

Compute Limitation

Face detection on the ESP32 uses the Viola-Jones or similar lightweight model at reduced resolution. It is CPU-bound, running on dual-core Xtensa LX6 at 240 MHz. Accuracy drops in poor lighting; false positives are common.

GPIO Scarcity

The AI Thinker module exposes very few usable GPIOs after the camera and SD card pins are consumed. GPIO0 and GPIO2 have boot strapping constraints. This limits how many peripherals can be added without an I2C/SPI expander.

2. Upgrade Opportunities

The following table maps high-value upgrade categories with their primary benefits and recommended timing.

Category	Key Opportunities	Priority
Vision	Face recognition, snapshot capture, night vision / IR LEDs, pan/tilt servo	**High**
Sensors	Replace HC-SR04 with VL53L0X ToF, add IR edge sensors, add PIR motion sensor	**High**
Motion / Robotics	Pan/tilt servo for camera, encoder feedback for odometry, patrol logic	**Medium**
Power System	LiPo battery + TP4056 charger, voltage regulation, battery level monitoring	**High**
Software Architecture	Modular firmware (camera, motion, sensor, alert modules), event queue	**Medium**
Communication / Control	Telegram bot alerts, WebSocket for low-latency control, mobile-friendly UI	**Medium**
Security & Reliability	Watchdog timer, auto-reconnect WiFi, SD card logging, rate-limited alerts	**Medium**
Platform Upgrade	Migrate to ESP32-S3 for larger PSRAM and better AI inference capability	**Advanced**

3. Specific Improvement Ideas

Each improvement is analyzed across seven dimensions: what it adds, why it matters, difficulty level, hardware changes, software changes, expected benefits, and expected tradeoffs.

3.1 Vision Upgrades

Face Recognition (vs. Detection Only)

What It Adds	Identifies specific enrolled individuals instead of triggering on any face
Why It Matters	Eliminates false alarms from family members; enables access-log by person
Difficulty	Intermediate — ESP32 can do recognition via ESP-WHO's face_recognition_mb_XXXX model
Hardware Changes	None required; optional: SD card to store enrolled face embeddings
Software Changes	• Enable face recognition pipeline in ESP-WHO (enroll mode + recognize mode) • Add HTTP endpoint to enroll new faces via snapshot • Store recognized name in alert payload
Expected Benefits	Dramatically reduces false positives; enables named alerts
Expected Tradeoffs	Adds ~30–50KB RAM usage; recognition latency adds ~200–400ms per frame; accuracy degrades in poor lighting or partial occlusion

Snapshot Capture on Detection

What It Adds	Saves a JPEG image of the triggering frame when face or motion is detected
Why It Matters	Evidence without manual monitoring; critical for a real security application
Difficulty	Beginner — camera_capture_jpg() is already available in ESP32 camera driver
Hardware Changes	Requires MicroSD card (already has slot on AI Thinker board)
Software Changes	• Initialize SD card via SPI on SDMMC pins • On detection event, call esp_camera_fb_get(), save raw JPEG to /sdcard/events/timestamp.jpg • Expose /list and /download HTTP endpoints
Expected Benefits	Complete audit trail; no cloud dependency; immediate portfolio value
Expected Tradeoffs	SD write latency 20–80ms per image; SDMMC and camera share SPI, must handle carefully

Pan/Tilt Servo Mount for Camera

What It Adds	Allows camera to rotate horizontally and vertically via two servo motors
Why It Matters	Turns the fixed-view camera into an active tracking system
Difficulty	Intermediate — requires free GPIOs and servo PWM control

Hardware Changes	• 2× SG90 or MG90S servo motors • 3D-printed or off-the-shelf pan/tilt bracket • GPIO expander (PCF8574 via I2C) to free up additional pins if needed
Software Changes	• PWM servo control (LEDC peripheral on ESP32) • HTTP endpoints /pan?angle=X and /tilt?angle=Y • Optional: auto-track detected face centroid using bounding box feedback
Expected Benefits	180° horizontal + 90° vertical coverage; enables face-tracking mode
Expected Tradeoffs	Two additional GPIOs required; SG90 torque is low; tracking loop can oscillate without dampening

Night Vision / IR LED Ring

What It Adds	Illuminates dark scenes invisibly using 850nm IR LEDs that the OV2640 sensor can detect
Why It Matters	A security system that fails in the dark is not a security system
Difficulty	Beginner — simple GPIO-controlled transistor switch for LED array
Hardware Changes	• 4–8× 850nm IR LEDs in ring configuration • NPN transistor (e.g. 2N2222) + resistor for GPIO control • Current-limiting resistors per LED
Software Changes	• GPIO output to enable/disable IR ring • Auto-enable below ambient light threshold (if LDR sensor added) • Manual toggle via web UI
Expected Benefits	Functional 24/7 operation; very low cost (~$2–5); immediate visible upgrade
Expected Tradeoffs	OV2640 color rendition becomes monochrome in IR-only mode; LEDs add ~150–300mA current draw

3.2 Sensor Upgrades

Replace HC-SR04 with VL53L0X Time-of-Flight Sensor

What It Adds	Laser-based ToF distance sensing with ~2mm accuracy over 0–2m range
Why It Matters	• HC-SR04 has a 15° beam cone — unreliable for narrow obstacle detection • Ultrasonic fails on soft/angled surfaces and at close ranges • VL53L0X works reliably from 5cm to 200cm with I2C interface
Difficulty	Beginner — VL53L0X has excellent Arduino library (Pololu or STM official)
Hardware Changes	VL53L0X module (~$3), SDA/SCL to any available I2C pins
Software Changes	Replace pulse-based HC-SR04 code with i2c_read() from VL53L0X; same threshold logic applies

Expected Benefits	More reliable obstacle detection; frees GPIO2/16; enables faster polling (100Hz capable)
Expected Tradeoffs	2m max range (vs HC-SR04's ~4m); less effective outdoors in bright sunlight due to IR interference

IR Edge / Cliff Detection Sensors

What It Adds	TCRT5000 infrared reflectance sensors detect surface drop-offs (table edges, stairs)
Why It Matters	Without edge detection, the robot will drive off elevated surfaces — a fundamental safety gap
Difficulty	Beginner
Hardware Changes	2–4× TCRT5000 modules mounted on the underside frame, pointed at the floor
Software Changes	• Digital read: surface present = HIGH, edge = LOW • Integrate into movement control: halt and reverse on edge detection
Expected Benefits	Safe operation on elevated surfaces; enables indoor patrol without human monitoring
Expected Tradeoffs	Only works on surfaces within ~1cm of sensor; black surfaces may trigger false positives

PIR Motion Sensor (HC-SR501)

What It Adds	Passive infrared sensor detects heat signatures from humans at 3–7m range
Why It Matters	• Camera-based detection is only active when streaming; PIR works even in standby mode • Enables wake-on-motion: robot activates and begins streaming when PIR triggers
Difficulty	Beginner — single digital output pin
Hardware Changes	HC-SR501 or AM312 module, one GPIO pin, adjustable sensitivity/delay
Software Changes	• Interrupt-based detection on PIR pin • Wake from deep sleep or activate camera stream on positive PIR event • Combine with camera detection for two-factor alert confirmation
Expected Benefits	Dramatic power savings in standby; faster initial response than camera-based detection
Expected Tradeoffs	PIR cannot distinguish between humans and large animals; ~2–3 second warm-up period required

3.3 Motion / Robotics Upgrades

Autonomous Patrol Logic

What It Adds	The robot follows a pre-defined or randomized patrol route without human input
Why It Matters	Changes the classification from 'remote-controlled toy' to 'autonomous security agent'
Difficulty	Intermediate
Hardware Changes	Edge sensors required; optional: encoder wheels for odometry
Software Changes	• State machine: PATROL → OBSTACLE → ALERT → RETURN states • Waypoint-based navigation using timed motor commands or odometry • Integration with detection system: halt patrol on face/motion detection
Expected Benefits	Truly autonomous operation; significant portfolio differentiation
Expected Tradeoffs	Dead reckoning drifts without encoders; reliable room mapping requires more sensors

Replace L298N with DRV8833 or TB6612FNG Motor Driver

What It Adds	More efficient motor driver with lower voltage drop and built-in protection
Why It Matters	• L298N drops ~2V per motor — at 5V input, motors only see ~3V • L298N generates significant heat and wastes battery power • DRV8833 drops only ~0.4V and is significantly smaller
Difficulty	Beginner — same pin interface
Hardware Changes	Replace L298N module with DRV8833 or TB6612FNG breakout board
Software Changes	No changes required — same motor direction/PWM logic applies
Expected Benefits	~30% longer battery runtime; better motor torque; reduced heat; smaller footprint
Expected Tradeoffs	DRV8833 limited to 1.5A continuous (sufficient for small motors); TB6612FNG costs slightly more

3.4 Power System Upgrades

LiPo Battery + Dedicated 3.3V/5V Regulation

What It Adds	Stable regulated power from a rechargeable LiPo cell with proper charging circuit
Why It Matters	• USB power tethers the robot; direct battery without regulation causes brownouts as battery drains • ESP32 requires stable 3.3V; motors cause voltage spikes that corrupt the MCU
Difficulty	Intermediate
Hardware Changes	• 1S 18650 or LiPo cell (3000–5000mAh) • TP4056 charging module

	• MT3608 boost converter for stable 5V rail • AMS1117-3.3 or HT7333 for MCU 3.3V • 100–470µF decoupling capacitors on motor power rail
Software Changes	• Optional: ADC read on voltage divider to monitor battery level • Low-battery alert via buzzer or web UI
Expected Benefits	Fully wireless operation; stable power prevents crashes; rechargeable and sustainable
Expected Tradeoffs	LiPo requires careful handling (overcharge/overdischarge protection); adds design complexity

3.5 Software Architecture Upgrades

Modular Firmware Architecture

What It Adds	Splits monolithic code into independent modules with clean interfaces
Why It Matters	Current single-file approach becomes unmaintainable as features grow; modularity enables testing and reuse
Difficulty	Intermediate
Hardware Changes	None
Software Changes	• Create separate .cpp/.h files: camera.cpp, motion.cpp, sensors.cpp, webserver.cpp, alerts.cpp, sd_logger.cpp • Define event types (FACE_DETECTED, OBSTACLE_NEAR, MOTION_TRIGGERED) • Use FreeRTOS queues to pass events between tasks • Each module subscribes to relevant events
Expected Benefits	Clean GitHub repo; easier debugging; professional code structure for CV
Expected Tradeoffs	Initial refactor time investment; FreeRTOS overhead (~10KB RAM)

SD Card Event Logging

What It Adds	Appends timestamped event records (detections, alerts, errors) to a CSV or JSON log file on SD
Why It Matters	Audit trail is essential for any real security system; demonstrates data engineering awareness
Difficulty	Beginner
Hardware Changes	MicroSD card (already present on AI Thinker)
Software Changes	• Initialize SDMMC in 1-bit SPI mode • Write log entries: timestamp, event_type, confidence, duration • HTTP endpoint /logs to download or view the log file

Expected Benefits	Complete activity history; easy to parse and visualize; strong demo feature
Expected Tradeoffs	SD write latency; SD wear over time with frequent writes (mitigate with write buffering)

3.6 Communication / Control Upgrades

Telegram Bot Alerts

What It Adds	Sends real-time push notifications to your Telegram phone when a detection event occurs, optionally with a snapshot image
Why It Matters	Remote alerting is the defining feature that separates a toy from a real security system
Difficulty	Intermediate — requires internet access, bot token, HTTPClient library
Hardware Changes	None (requires internet-accessible WiFi router)
Software Changes	• Register Telegram bot via @BotFather, obtain token and chat_id • On detection: POST to api.telegram.org/bot{token}/sendPhoto with JPEG buffer • Rate-limit alerts (minimum 30s between messages to prevent spam)
Expected Benefits	Remote monitoring from anywhere; image evidence with notification; zero-cost platform
Expected Tradeoffs	Requires internet connectivity; Telegram API changes can break integration; latency ~1–3s

WebSocket for Real-Time Control

What It Adds	Replaces HTTP polling with a persistent WebSocket connection for motor commands
Why It Matters	HTTP GET requests for motor control introduce ~50–200ms latency per command; WebSocket achieves <10ms
Difficulty	Intermediate
Hardware Changes	None
Software Changes	• Use ESPAsyncWebServer with AsyncWebSocket plugin • Client sends JSON commands: {cmd: 'move', dir: 'forward', speed: 200} • Bidirectional: server pushes sensor data back to UI in real time
Expected Benefits	Responsive control; real-time sensor dashboard in browser; modern architecture
Expected Tradeoffs	More complex server code; connection state must be managed; single client at a time

3.7 Platform Upgrade

Migrate to ESP32-S3 (When Needed)

What It Adds	Xtensa LX7 dual-core at 240MHz, 512KB SRAM, support for up to 16MB PSRAM, USB OTG, and a vector extension unit for neural network inference
Why It Matters	• ESP32-S3 can run TensorFlow Lite Micro models on-device • More PSRAM enables higher-resolution processing and more concurrent tasks • AI Thinker ESP32-S3-WROOM modules are available
Difficulty	Advanced — requires re-porting the firmware
Hardware Changes	New development board (ESP32-S3-DevKitC or custom PCB); OV2640 camera still compatible
Software Changes	• Port from ESP-IDF v4 to v5 if needed • Update camera driver for S3 • Optionally integrate TFLite Micro for on-device model inference
Expected Benefits	Future-proof platform; enables true ML inference; better AI portfolio story
Expected Tradeoffs	Board cost is higher; porting effort is significant; not necessary until hitting ESP32-CAM limits

4. Recommended Upgrade Roadmap

Phase 1: Foundation — Easy, High-Impact Wins

Goals

- Add snapshot capture on detection
- Add SD card event logging
- Add IR edge/cliff detection sensors
- Replace L298N with DRV8833
- Add night vision IR LED ring

Components

- MicroSD card (already on board)
- 2–4× TCRT5000 edge sensor modules (~$3)
- DRV8833 motor driver module (~$4)
- 4× 850nm IR LEDs + NPN transistor (~$2)

Coding Tasks

- SD initialization + JPEG save on detection event
- CSV log writer for events
- Edge sensor digital read integration
- Motor driver pin remapping (minimal change)
- IR LED GPIO control

Risk Level: Low — all changes are additive or direct replacements

Difficulty: Beginner to Low-Intermediate

Why It Matters: These changes deliver immediate visible value (evidence capture, safer operation, longer runtime) with almost no architectural disruption. Each one is independently deployable.

Phase 2: Intelligence — Stronger Sensing & Alerting

Goals

- Enable face recognition (not just detection)
- Add Telegram push alerts with snapshot images
- Replace HC-SR04 with VL53L0X ToF sensor
- Add PIR sensor for standby wake-on-motion
- Add WebSocket control interface
- Implement stable LiPo power system

Components

- VL53L0X ToF module (~$4)
- HC-SR501 PIR sensor (~$2)
- 18650 LiPo cell + TP4056 + MT3608 boost converter (~$8 total)
- No additional IC needed for Telegram (software only)

Coding Tasks

- ESP-WHO face recognition pipeline integration
- Face enrollment HTTP endpoint
- Telegram bot integration with HTTPClient
- VL53L0X I2C driver integration

- PIR interrupt handler + deep sleep wake logic
- ESPAsyncWebServer + WebSocket handler
- ADC battery level reader

Risk Level: Medium — face recognition and power redesign require careful testing

Difficulty: Intermediate

Why It Matters: This phase transforms the robot from a demonstration device into a functional security tool. Remote alerting + face recognition + continuous power are the three features that define a real system.

Phase 3: Advanced — Autonomous, Portfolio-Grade Version

Goals

- Implement autonomous patrol with state machine
- Add pan/tilt servo with optional face-tracking
- Refactor firmware into modular FreeRTOS architecture
- Add mobile-responsive web dashboard with live sensor telemetry
- Optional: migrate to ESP32-S3 for TFLite Micro inference

Components

- 2× SG90 servo motors + pan/tilt bracket (~$8)
- PCF8574 I2C GPIO expander if needed (~$2)
- ESP32-S3 board (optional upgrade, ~$15)

Coding Tasks

- Patrol state machine: IDLE → PATROL → OBSTACLE → ALERT → RETURN
- Servo PWM (LEDC) with pan/tilt control endpoints
- Face centroid tracking loop (bounding box → servo position)
- Modular refactor: camera.cpp, motion.cpp, sensors.cpp, webserver.cpp, alerts.cpp, sd_logger.cpp
- FreeRTOS event queue for inter-module communication
- React or vanilla JS responsive dashboard with WebSocket telemetry

Risk Level: Medium-High — patrol logic requires physical testing; S3 migration is a significant effort

Difficulty: Advanced

Why It Matters: Phase 3 produces a system that is genuinely impressive to engineers and recruiters: autonomous navigation, active tracking, clean modular codebase, and real-time dashboard. This is the portfolio version.

5. Best Next Step For You

Based on the current state of the project, the following three upgrades are the highest-impact, most feasible next steps — ordered by recommended sequence.

#	Upgrade	Why Now
1	Snapshot Capture on Detection	The SD card slot is already on your board. Saving a JPEG on each detection event takes ~20 lines of code and immediately transforms your project from an alert-only system to an evidence-collection system. This is the single highest ROI change available right now.
2	Telegram Alert with Image	Once snapshots exist on SD, sending them via Telegram HTTPClient is a natural follow-up. This enables remote monitoring — the core value proposition of any security system. You go from 'works at home on the local network' to 'works anywhere in the world'. Intermediate difficulty, zero hardware cost.
3	PIR Sensor + Wake-on-Motion	Adding a $2 HC-SR501 and a single GPIO interrupt enables your robot to sleep when idle and wake automatically when a human enters the room. This makes the system behave like real security hardware rather than a device that must be manually activated.

6. Decision Matrix

This matrix compares all upgrade options across seven criteria to support prioritization decisions.

Upgrade	Difficulty	Cost	Impact	HW Changes	SW Changes	Recommend
Snapshot on Detection	Beginner	Free ($0)	Very High	None	20 lines	Now
Telegram Alerts	Intermediate	Free ($0)	Very High	None	~100 lines	Now
PIR Motion Sensor	Beginner	~$2	High	1 sensor + GPIO	30 lines	Now
IR Night Vision LEDs	Beginner	~$3	High	LEDs + transistor	10 lines	Now
SD Card Logging	Beginner	Free ($0)	High	None (on-board)	50 lines	Now
Replace L298N → DRV8833	Beginner	~$4	Medium	New driver board	Minimal rewire	Soon
IR Edge Sensors	Beginner	~$3	High	2–4 sensors	40 lines	Soon
VL53L0X ToF Sensor	Beginner	~$4	Medium	New sensor, I2C	Replace HC-SR04	Soon
Face Recognition	Intermediate	Free ($0)	Very High	SD for embeddings	ESP-WHO pipeline	Soon
WebSocket Control	Intermediate	Free ($0)	Medium	None	Refactor server	Later
Pan/Tilt Servo	Intermediate	~$8	High	2 servos + mount	LEDC PWM + UI	Later
LiPo Power System	Intermediate	~$10	High	Full power circuit	ADC monitor	Later
Autonomous Patrol	Advanced	~$5	Very High	Edge sensors	State machine	Phase 3
Modular FW Architecture	Intermediate	Free ($0)	High	None	Full refactor	Phase 3
ESP32-S3 Migration	Advanced	~$15	High	New board	Full port	Phase 3

7. Architecture Improvement Notes

The current project is likely structured as a single large .ino or .cpp file. As features grow, this becomes difficult to maintain. The recommended target architecture uses FreeRTOS tasks and an event queue to decouple modules.

7.1 Recommended Module Structure

Module	Responsibilities	Key APIs / Interfaces
`camera.cpp`	Initialize OV2640, capture frames, expose frame buffer to other modules	camera_init(), camera_capture_fb(), camera_release_fb()
`detection.cpp`	Face detection / recognition pipeline, bounding box output	detection_init(), detection_run(fb), detection_get_result()
`motion.cpp`	Motor control, direction, speed, patrol state machine	motion_init(), motion_drive(dir, speed), motion_set_state()
`sensors.cpp`	VL53L0X distance, PIR interrupt, edge sensors, battery ADC	sensors_init(), sensors_get_distance(), sensors_get_pir()
`webserver.cpp`	HTTP + WebSocket server, route handlers, stream endpoint	webserver_init(), webserver_broadcast_event(event)
`alerts.cpp`	Telegram bot sender, buzzer trigger, alert rate limiter	alerts_init(), alerts_send(event, fb)
`sd_logger.cpp`	SD mount, JPEG save, CSV event log, download endpoint	sdlog_init(), sdlog_save_image(fb), sdlog_append_event(e)
`main.cpp`	FreeRTOS task creation, event queue, system init sequence	app_main(), event_queue xQueueCreate()

7.2 Event-Driven Communication Pattern

All modules communicate via a central FreeRTOS event queue. This eliminates tight coupling between modules.

Event Flow Example

PIR triggers → sensors.cpp posts EVENT_MOTION_DETECTED to queue → main.cpp receives event → calls camera_capture_fb() → calls detection_run(fb) → if FACE_DETECTED: calls alerts_send() + sdlog_save_image() → webserver_broadcast_event() updates live dashboard

7.3 FreeRTOS Task Layout

- Task 1 — CameraStream: Continuous MJPEG frame push to active WebSocket clients (Core 0, high priority)
- Task 2 — Detection: Periodic face detection on captured frames (Core 1, medium priority)
- Task 3 — Sensors: Poll VL53L0X + PIR + battery ADC every 50ms (Core 0, low priority)

- Task 4 — WebServer: Handle HTTP requests and WebSocket messages (Core 0, medium priority)
- Task 5 — AlertSender: Consume alert events from queue, send Telegram (Core 1, low priority)

8. Portfolio / CV Value

As currently built, this project demonstrates: basic ESP32 programming, WiFi streaming, and simple motor control. These are solid foundations but do not stand out at the portfolio level. The following additions transform it into a genuinely impressive systems project.

8.1 Features That Signal Engineering Maturity

- Modular firmware architecture with FreeRTOS tasks: shows knowledge of RTOS, concurrency, and embedded software design patterns
- Event-driven alerting with image evidence: shows end-to-end system thinking, not just hardware wiring
- SD logging with downloadable event history: shows data persistence and API design awareness
- Autonomous patrol with obstacle avoidance: shows state machine design and robotic systems experience
- Clean GitHub README with: system architecture diagram, demo video, hardware BOM, setup instructions, and code documentation

8.2 What Makes It Look Production-Like

- Proper power design (regulated rails, decoupling, battery monitoring) — not just USB power
- Rate-limited alerting with cooldown logic — shows awareness of real-world operational constraints
- Error handling and watchdog timer — shows reliability engineering mindset
- A mobile-responsive web dashboard — shows full-stack awareness beyond the embedded layer
- Unit-testable module boundaries — even without tests, the structure signals professional intent

8.3 GitHub Presentation Strategy

- Repository name: esp32-security-robot (clear, searchable)
- README sections: Project Overview → Architecture Diagram → Feature List → Hardware BOM → Wiring Diagram → Setup Instructions → Demo Video
- Include a /docs folder with the architecture diagram as an SVG or PNG
- Tag releases: v1.0-basic-stream, v2.0-face-detection, v3.0-autonomous
- Add GitHub Actions badge (even a simple build check adds credibility)

8.4 LinkedIn / CV Description Template

Example CV Entry

ESP32-CAM Autonomous Security Robot (2025–2026) Built a WiFi-connected mobile surveillance system on ESP32-CAM with autonomous patrol, face recognition, real-time

Telegram alerting with image capture, and FreeRTOS-based modular firmware. Designed complete power system with LiPo management, VL53L0X ToF sensing, and WebSocket control interface. Stack: C++, ESP-IDF, FreeRTOS, ESP-WHO, REST/WebSocket APIs.

Best Practical Evolution Plan for This Robot

This final section consolidates the entire document into one actionable summary for direct execution.

Start Here — This Week

Add snapshot capture to SD card. Open the AI Thinker board, confirm the SD slot is populated with a card, initialize SDMMC in SPI mode, and call esp_camera_fb_get() on every face detection event. Save to /sdcard/events/YYYYMMDD_HHMMSS.jpg. Verify with a simple /list HTTP endpoint that returns filenames. Estimated time: 2–4 hours.

Week 2 — Remote Alerting

Add Telegram bot alerting. Register a bot, get your chat_id, and use ESP32's HTTPClient to POST to the Telegram sendPhoto API using the buffered JPEG from step 1. Add a 30-second cooldown between alerts. Test from outside your home network. Estimated time: 3–5 hours.

Week 3–4 — Sensing Reliability

Add a PIR sensor for wake-on-motion and replace HC-SR04 with VL53L0X ToF sensor via I2C. The PIR costs $2 and adds one GPIO interrupt. The VL53L0X replaces two uncertain GPIO pins with a single reliable I2C bus. Also add IR edge sensors if the robot will operate on elevated surfaces. Estimated time: 4–6 hours total.

Month 2 — Intelligence Layer

Enable face recognition using ESP-WHO's recognition pipeline. Add face enrollment endpoint. Update Telegram alerts to include the recognized name (or 'Unknown' for strangers). Add CSV event logging to SD. Refactor motor driver to DRV8833 and begin modularizing the firmware. Estimated time: 8–12 hours.

Month 3 — Portfolio Version

Implement autonomous patrol state machine. Add pan/tilt servo with face-tracking. Implement LiPo power system. Refactor firmware into full modular FreeRTOS architecture. Add mobile-responsive dashboard with WebSocket telemetry. Write comprehensive README with architecture diagram, demo video, and BOM. Tag release as v3.0-autonomous. Estimated time: 20–30 hours.

The robot you have today is a solid v1.0. Every upgrade listed here is achievable in isolation — you do not need to build phase 3 before phase 1 works. Start small, ship often, and document what you build. The system will grow into something genuinely impressive.